1-A) a resposta para a alinea A é só que como temos de correr cada valor e verificar se é maior do que o valor maior anterior, e se for considerar o novo maior, ent no pior dos cabos o maior é o ultimo por ordem crescente, portanto na pior das hipoteses, O(N) já que tem de passar por todos os N numeros.

Lab03-src > 1_1 > C 1_1a.c > ...

```c
#include <stdlib.h>
#include <stdio.h>
// só pra teste
int V[] = {1, 3, 5, 8, 9, 6, 2};
int findMaxUniM_otimizado(int *V, int N) {
    int esq = 0;
    int dir = N - 1;
    int meio;

    // enquanto houver mais de 1 elemento
    while (esq < dir) {
        meio = (esq + dir) / 2;

        // compara com o da frente pra ver se ta a subir
        if (V[meio] < V[meio + 1]) {
            // ainda sobe, maximo ta pra direita
            esq = meio + 1;
        } else {
            // ta a descer, maximo ta na esquerda ou e este
            dir = meio;
        }
    }

    // no fim o esq e dir ficam no mesmo sitio (pico)
    return V[esq];
}

int main() {
    // calcula o tamanho do array N
    int N = sizeof(V) / sizeof(V[0]);

    // chama a funcao e guarda o resultado
    int maximo = findMaxUniM_otimizado(V, N);

    // imprime o resultado pa ver se funcionou
    printf("o valor maximo e: %d\n", maximo);

    return 0;
}
```

```
archlinux% gcc 1_1a.c -o teste
archlinux% ./teste
o valor maximo e: 9
archlinux%
```

1-C) no codigo da alinea b) é O(log N) pq como dentro do while tamos sempre a calcular o meio e a deitar fora metade dos numeros que nao interessam, ele em vez de passar pelos N numeros um a um vai sempre cortando o problema a meio a cada iteração. ent no pior dos cabos a quantidade de cortes q ele faz ate achar o pico é só o logaritmo de N, portanto fica O(log N).

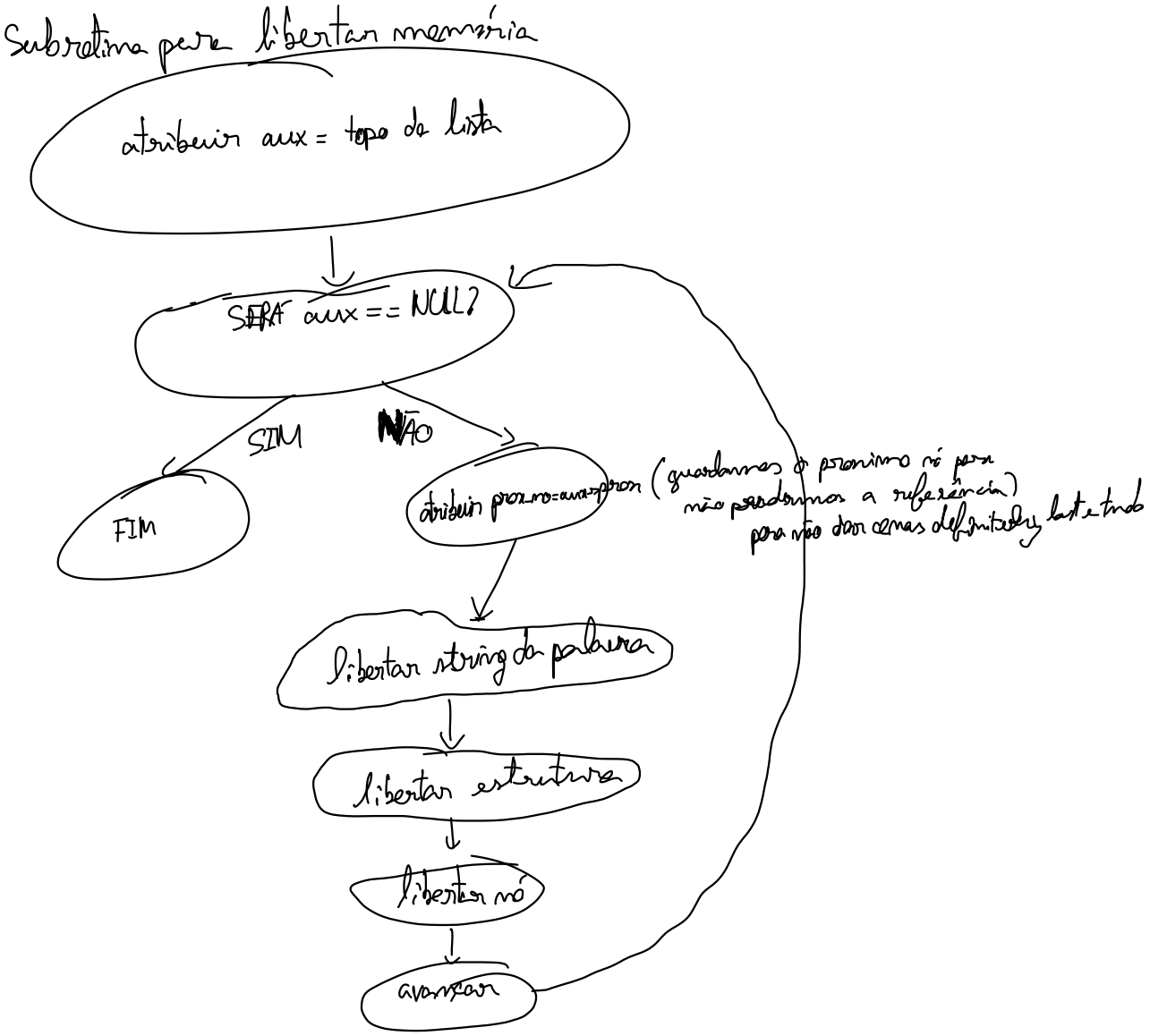
2-b) código ta no github
2-c)

a gestao de memoria é ineficiente pq ele vai ver o tamanho da maior palavra e dps aloca esse tamanho maximo pra todas as strings, mesmo q sejam mt pequenas. para alem disso, ele aloca logo memoria pro numero total de palavras q tao no ficheiro, oq desperdica bue espaco se houver bues palavras repetidas. a solucao é fazer alocacao so dps, ou seja, alocamos o espaco para letras so qnd confirmamos q a palavra é nova, e usamos so o tamanho exato dessa palavra (strlen +1). dps, sobre o fluxograma, é so tirar a caixa de alocar caracteres que ta na subrotina de alocar e meter uma caixa nova na subrotina de preencher tabela, mesmo a seguir à decisao "palavra é nova? Sim", a dizer pra alocar a memoria po tamanho exato da palavra.

2-a) n é assim nada demais, simplesmente é descrevermos que, como devíamos ter feito no eliza usa-se o header para as funcionalidades da cena e dps ta tudo definido direitinho no .c. Agora, se é só isto que querem na pergunta eu ns, mas ya. o amigo diz que isto convém pq permite-te teres mudanças do tipo de estrutura de dados que usas (tipo podes dps usar outros algoritmos e o main.c n fica fdd porque tens na mesma os acessos certos no .h, pq ele usa funções tipo acederLista (é um exemplo) para aceder aos resultados dados pelo .c. isto tá meio confuso mas se precisares explico-te melhor

2-b) efetivamente a mesma lenga lenga, portanto podiamos mudar o words.c pra usar tabelas em vez de listas e o main.c ia continuar a funcionar igual sem termos de mudar cenas lá. acho q percebes a ideia perchance





pra alinea e):
basicamente como o words.h se mantem igual, o main.c nem nota a diferenca. o q tinhamos de fazer pra criar o words_tables.c era logo tirar o #include "list.h" lá de cima. depois, em vez de usarmos o ponteiro da lista t_lista *wordsListPt;, trocavamos pra um array dinamico tipo t_words **tabela e criavamos mais uma variavel pra controlar a capacidade maxima da tabela. na funcao wordsCreateData() faziamos o malloc inicial da tabela em vez de chamar o iniLista(). no wordsAddWord() em vez de andarmos a saltar de node em node com o getProxElementoLista(), faziamos um ciclo for normal pelo array pra ver se a palavra ja la tava. e pronto, no fim o wordsFreeData() levava um ciclo for a dar free a todos os elementos da tabela em vez de usar a funcao libertaLista().

finalmente o 2.3 (da discussao comparativa):
em termos de memoria, a lista ligada aloca sempre o espaco exato q precisamos à medida q chegam palavras novas, mas tem o overhead de gastar uns bytes extra em cada palavra por causa do ponteiro prox. a tabela n tem os ponteiros extra, mas pode desperdicar bue memoria se alocarmos um array gigante q dps fica a meio.
em termos de complexidade de tempo, vao dar ao mesmo. como as palavras tao atiradas la pra dentro ao calhas e nada ta ordenado alfabeticamente, o tempo de procurar se uma palavra ja existe é $O(N)$ tanto na tabela como na lista ligada, pq na pior das hipoteses tens de ler a cena ate ao fim.